

Advanced NLP

Session 10 - Parameter-Efficient Fine-Tuning (PEFT: LoRA, QLoRA) & Distillation

Teacher: Dr. Behrouz Minaei-Bidgoli
Presenter: Saba Sattari
Presentation creator: Saba Sattari

Iran University of Science & Technology

Contents

1. Parameter-Efficient Fine-Tuning(PEFT)
2. Knowledge Distillation

1.

Parameter-Efficient Fine-Tuning(PEFT)

Main Resource For this Section:
CS224N: Natural Language Processing with Deep Learning Stanford / Winter 2026

Why fine-tuning only some parameters?

High Computational Cost

Impractical Full Fine-Tuning:

- Fine-tuning all parameters is impractical with large models.

Resource Bottlenecks:

- Updating billions of weights creates extreme compute, GPU memory, and storage demands for each task.

Model Overparameterization

Intrinsic Redundancy:

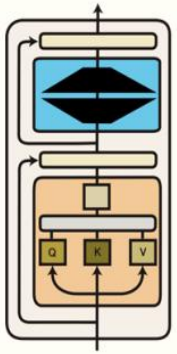
- State-of-the-art models are massively overparameterized.

Performance Parity:

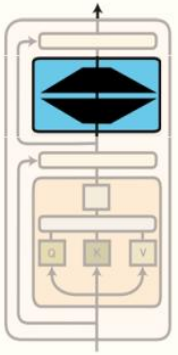
- Parameter-efficient fine-tuning matches performance of full fine-tuning.

PEFT Performance ≈ Full Fine-Tuning

Achieved with a small fraction of trainable parameters

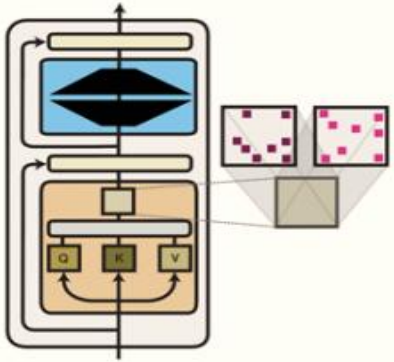


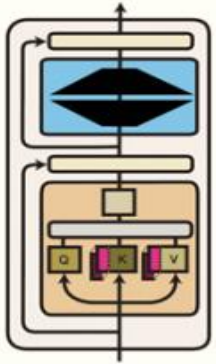
Full Fine-tuning
Update **all model parameters**

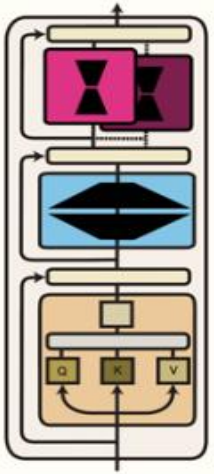


Parameter-efficient Fine-tuning
Update a **small subset** of model parameters

Different perspectives to think about PEFT







1. Parameter Level

Selective Updates

Focus: Model Weights
Selective vs. Additive parameters

2. Input Level

Prompt Tuning

Prefix Tuning

Focus: Representation Space
Virtual tokens & key-value prefixes

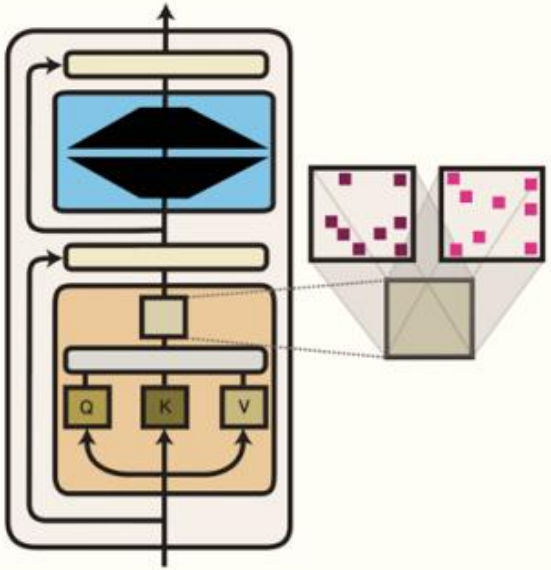
3. Function Level

Adapter Modules

Focus: Network Architecture
Low-rank matrices & bottleneck layers

https://www.modulardeeplearning.com/

A Parameter Perspective of Adaptation



Sparse Subnetworks

Weight Masking & Selection

Identifies and trains only a critical subset of parameters while freezing the rest of the base model architecture.

BitFit & Subnetwork Isolation

Focuses fine-tuning exclusively on bias vectors or sparse subnetworks,

Low-rank Composition

Matrix Factorization (LoRA)

Decomposes heavy weight update matrices into low-rank representations to minimize trainable parameter counts.

Structured Parameter Efficiency

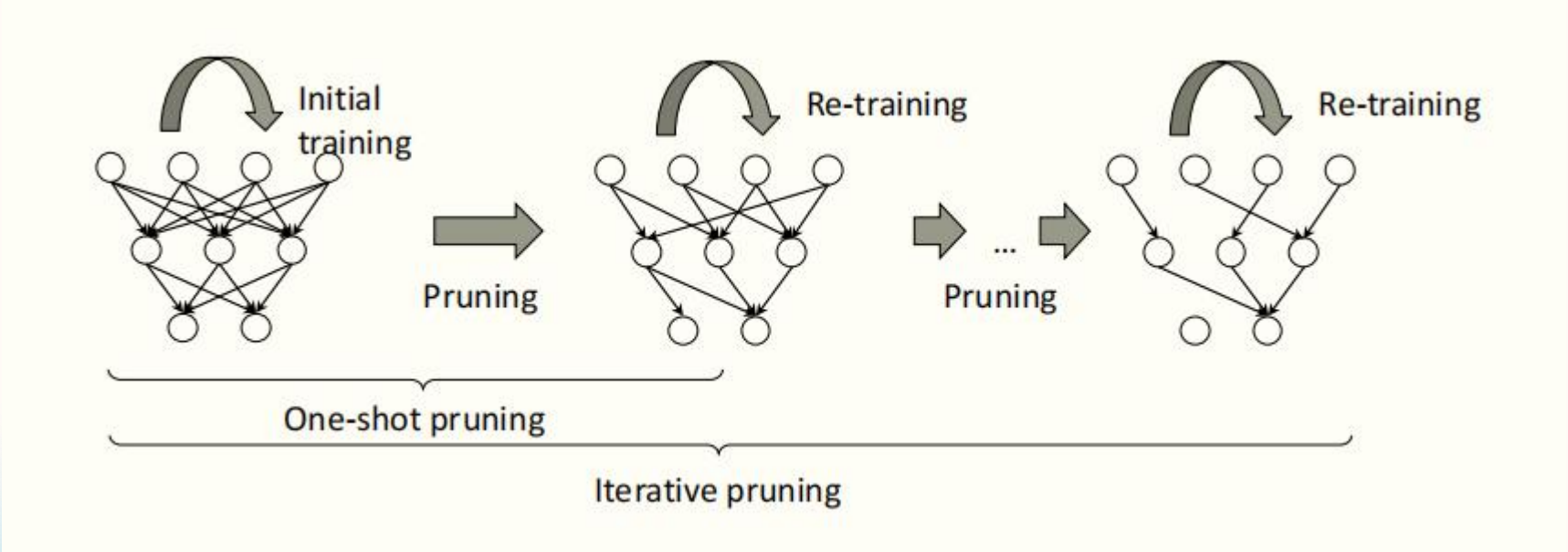
Constrains updates to intrinsic low dimensions, ensuring fast adaptation without modifying original network weights.

Sparse subnetworks

- A common inductive bias on the module parameters is **sparsity**
- Most common sparsity method: **pruning**
- Pruning can be seen as applying a binary mask b in {0, 1}^d that selectively keeps or removes each connection in a model and produces a subnetwork.
- Pruning criteria for determining the mask:
  - First-order / Magnitude: weight magnitude |w|
  - Second-order / Sensitivity: Fisher Information / Gradient-based

Pruning

- During pruning, a fraction of the lowest-magnitude weights are removed
- The non-pruned weights are re-trained
- Pruning for multiple iterations is more common



Pruning and Binary Mask

- We can also view pruning as adding a task-specific vector ϕ to the parameters of an existing model $f_{\theta'} = f_{\theta+\phi}$ where $\phi_i = 0$ if $b_i = 0$

- If the final model should be sparse, we can multiply the existing weights with the binary mask to set the pruned weights to 0: $f_{\theta'} = f_{\theta \circ b+\phi}$. These weight values were moving to 0 anyway [Zhou et al., 2019]

Element-wise product (Hadamard product)

- **Diff pruning:** we can perform pruning only based on the magnitude of the module parameters ϕ rather than the updated $|\theta + \phi|$ parameters [Guo et al., 2021]

The Lottery Ticket Hypothesis

- Dense, randomly-initialized models **contain subnetworks** (“winning tickets”) that—when trained in isolation—**reach test accuracy comparable to the original network** in a similar number of iterations

- Has also been verified in RL and NLP and for larger models in computer vision [Frankle]

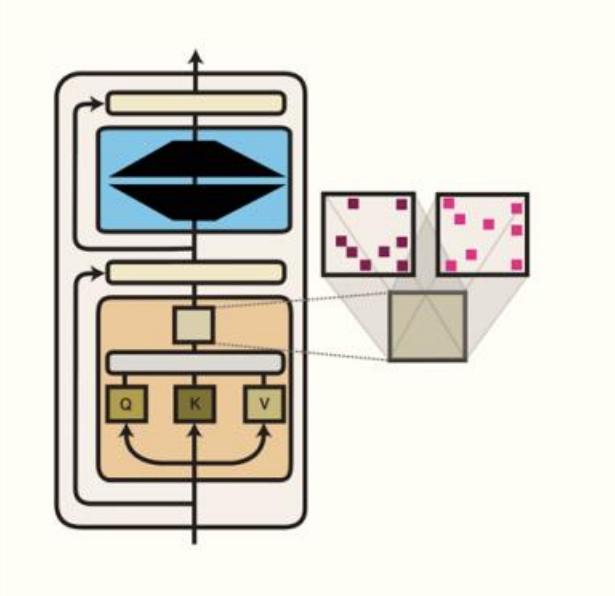
- Prior work [Chen; Prasanna] has found winning tickets in pre_trained models such as BERT
  . Sparsity ratios: from 40% (SQuAD) to 90% (QQP and WNLI)

- Subnetworks trained on a general task like masked language modelling **transfer** best

A Parameter Perspective of Adaptation

✓ Sparse Subnetworks

• Low-rank Composition



Revisit the full fine-tuning

Pre-trained Model Setup

Autoregressive Language Model

Assume a pre-trained model $P_\phi(y \mid x)$, such as GPT based on Transformer architecture.

Downstream Adaptation

Adapt the pre-trained weights to specific target tasks (e.g., summarization, QA).

Training Dataset Structure

Given a dataset of context-target pairs $\{(x_i, y_i)\}_{i=1,\ldots,N}$.

Base Model Parameterization

Initial Parameters: $\phi\ o$ | Target: $P\ \phi\ (y\ |x\ )$

Full Fine-Tuning Optimization

Full Parameter Update

During full fine-tuning, initial weights ϕ_0 are updated to $\phi_0 + \Delta\phi$
across all model layers.

Gradient-Based Optimization

Follows the gradient vector to maximize the conditional language modeling likelihood objective.

High Memory Overhead

Requires storing full gradients, optimizer states, and updating every parameter $\Delta\phi$.

$$\max_{\phi} \sum \sum^{|y|} \log\left(P_\phi(y_t \mid x, y_{<t})\right)$$

LoRA: low rank adaptation (Hu et al., 2021)

- For each downstream task, we learn a different set of parameters $\Delta\phi$
- $|\Delta\phi| = \phi\ o$
- GPT-3 has a $|\phi\ o|$ of 175 billion
  - Expensive and challenging for storing and deploying many independent instances

- Can we do better?

- **Key idea:** encode the task-specific parameter increment $\Delta\phi = \Delta\phi(\Theta)$ by **a smaller sized set of parameters Θ** , $\Theta \ll |\phi\ o|$
- The task of finding $\Delta\phi$ becomes optimizing over Θ

$$\max_{\Theta} \sum_{(x,y)} \sum_{t=1}^{|y|} \log\left(P_{\phi_0+\Delta\phi(\Theta)}(y_t \mid x, y_{<t})\right)$$

Low-rank-parameterized update matrices

- Updates to the weights have a low "intrinsic rank" during adaptation (Aghajanyan et al. 2020)

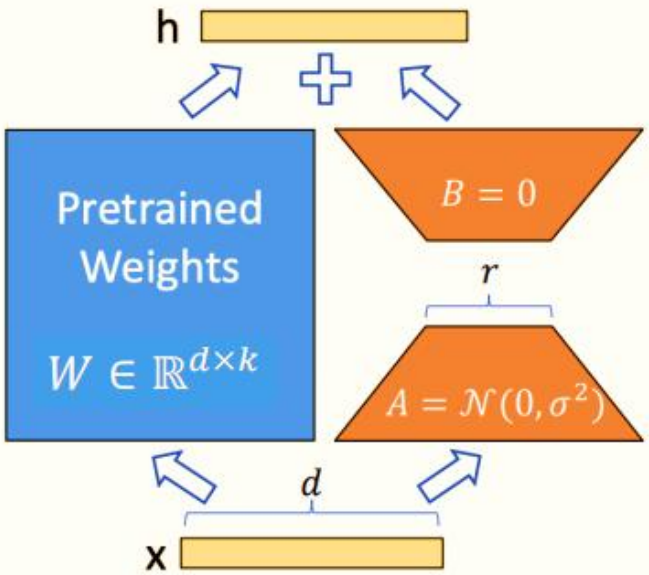
- $W_0 \in \mathbb{R}^{d \times k}$: a pretrained weight matrix
- Constrain its update with a low-rank decomposition:

$$W_0 + \Delta W = W_0 + \alpha BA$$

$$\text{where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

- α is the tradeoff between pre-trained "knowledge" and task-specific "knowledge"

- Only A and B contain **trainable** parameters

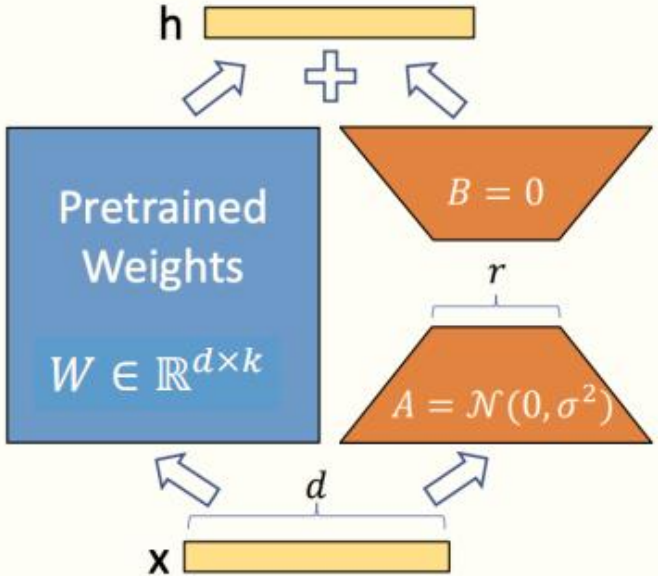


Low-rank-parameterized update matrices

• As one increase the number of trainable parameters, training LoRA converges to training the original model

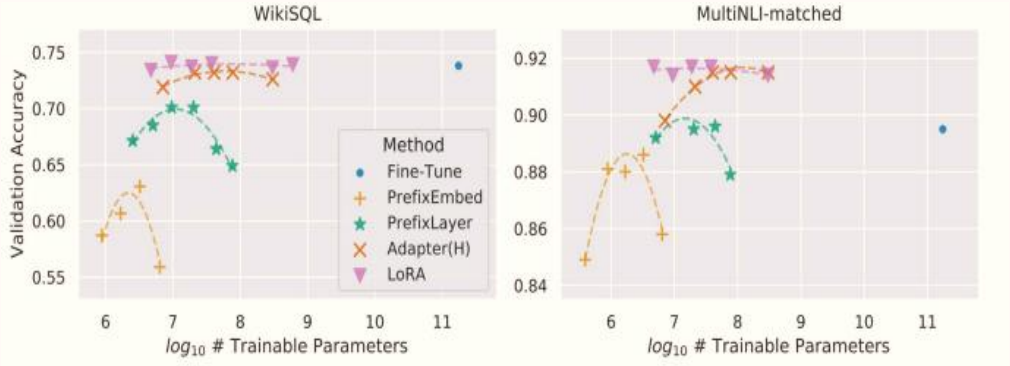
• **No additional inference latency:** when switching to a different task, recover W_0 by subtracting BA and adding a different $B'A'$

• Often LoRA is applied to the weight matrices in the self-attention module



LoRA in practice: scaling up to GPT-3 175B

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($Adapter^H$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($Adapter^H$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1



LoRA matches or exceeds the fine_ tuning baseline on all three datasets

LoRA exhibits better scalability and task performance

Understanding low-rank adaptation

Which weight matrices in Transformers should we apply LoRA to?

	# of Trainable Parameters = 18M						
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Rank r	8	8	8	8	4	4	2
WikiSQL (±0.5%)	70.4	70.0	73.0	73.2	71.4	**73.7**	**73.7**
MultiNLI (±0.1%)	91.0	90.8	91.0	91.3	91.3	91.3	**91.7**

Adapting both Wq and Wv gives the best performance overall.

What is the optimal rank r for LoRA?

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL(±0.5%)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI (±0.1%)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

LoRA already performs competitively with a very small r

From LoRA to QLoRA

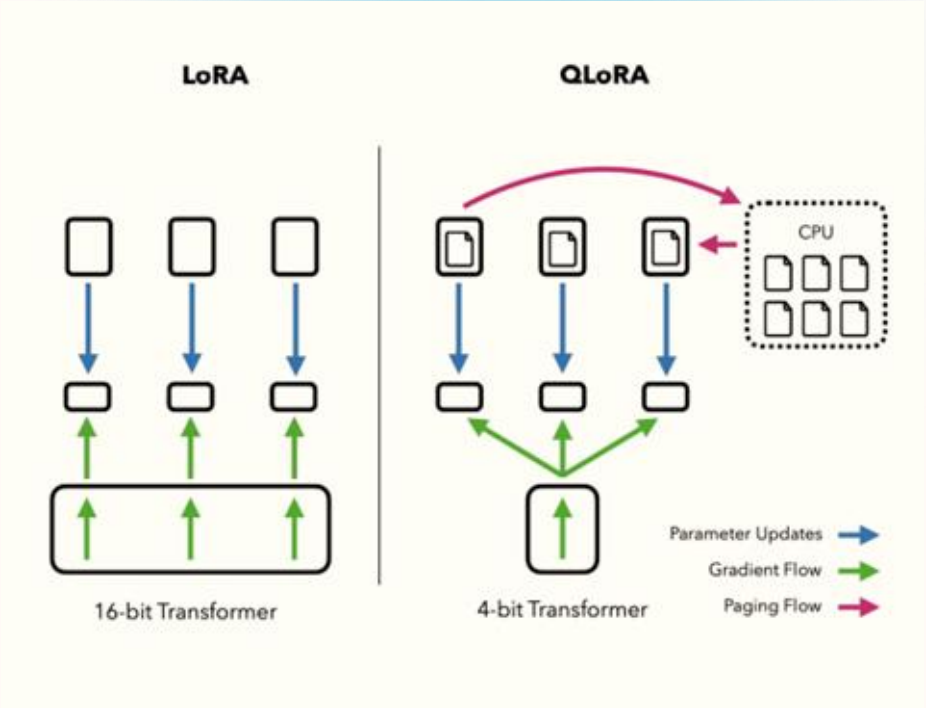
Significantly reduces GPU memory overhead while preserving full 16-bit fine-tuning performance.

4-Bit Precision

Quantizes the base transformer model down to 4-bit precision to drastically cut memory requirements.

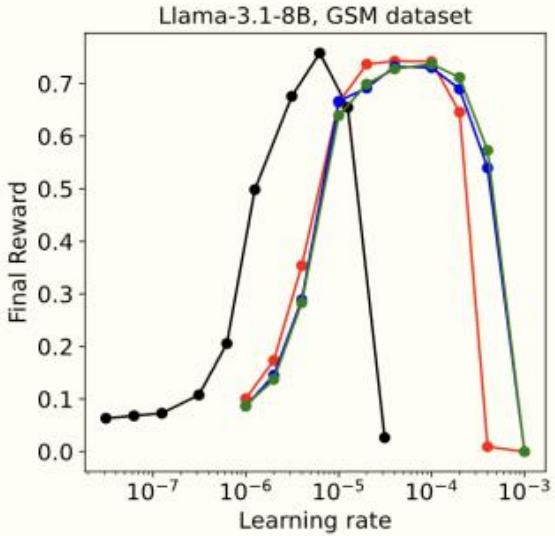
Paged Optimizers

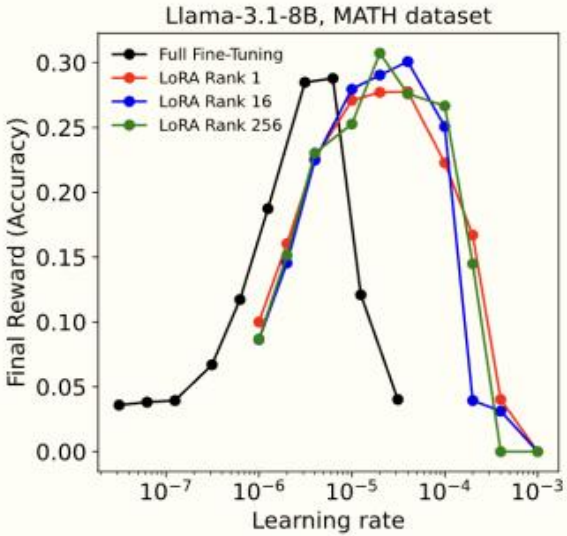
Leverages paged optimizers to seamlessly handle memory spikes and prevent out-of-memory errors.



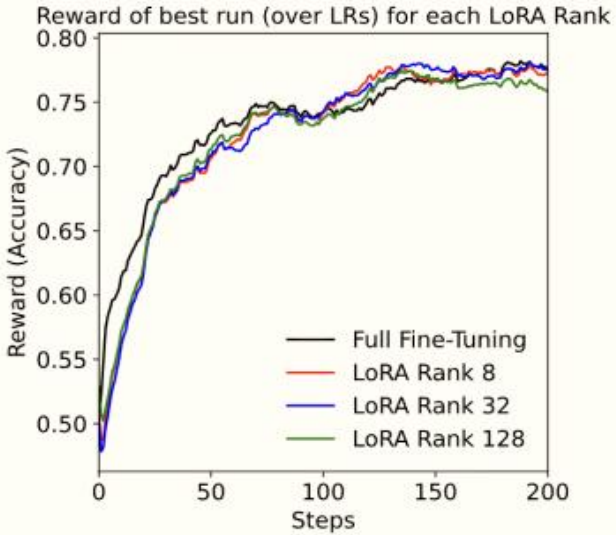
Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "Qlora: Efficient finetuning of quantized llms." arXiv preprint arXiv:2305.14314 (2023).

LoRA matches full fine tuning for RL, even with rank as low as 1





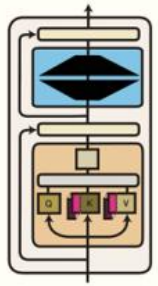
LoRA shows a wider range of performant learning rates and arrives at the same peak performance as FullFT



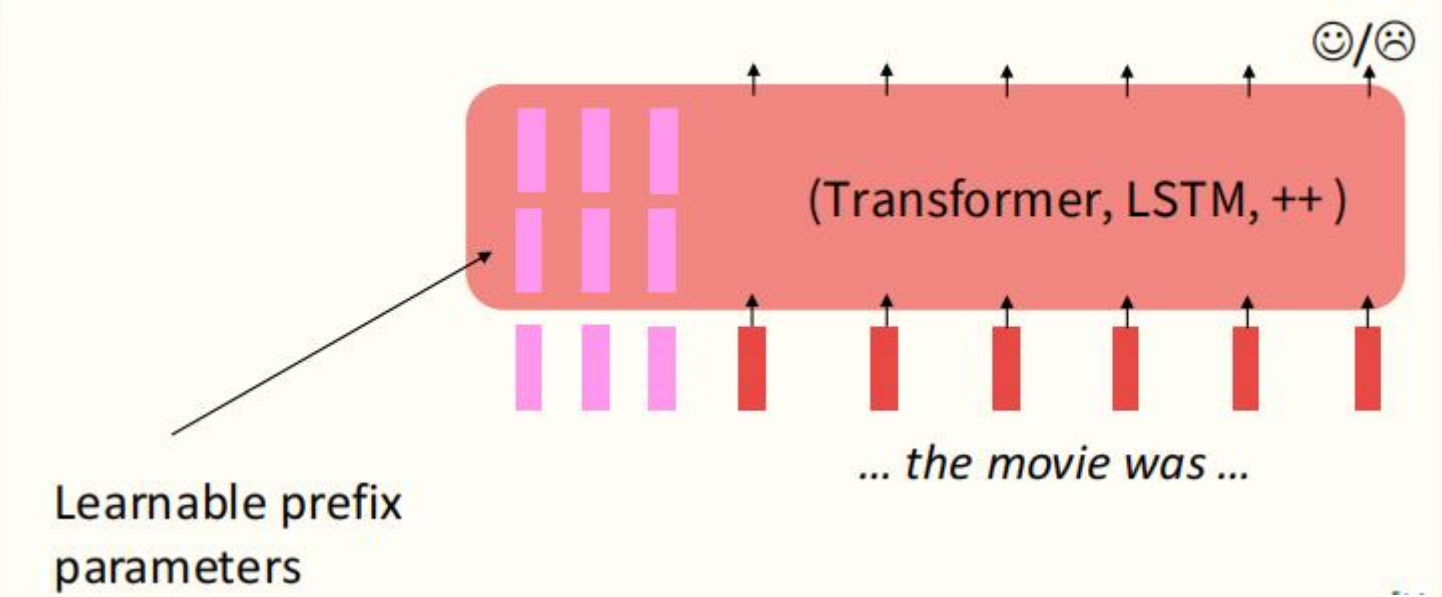
DeepMath with Qwen3-8b-base. The learning curve for different ranks and full fine-tuning

LoRA from Scratch → Your First QLoRA Run

https://arxiv.org/abs/2305.14314



An input perspective of adaptation



[Li and Liang , Lester]

Prefix-Tuning

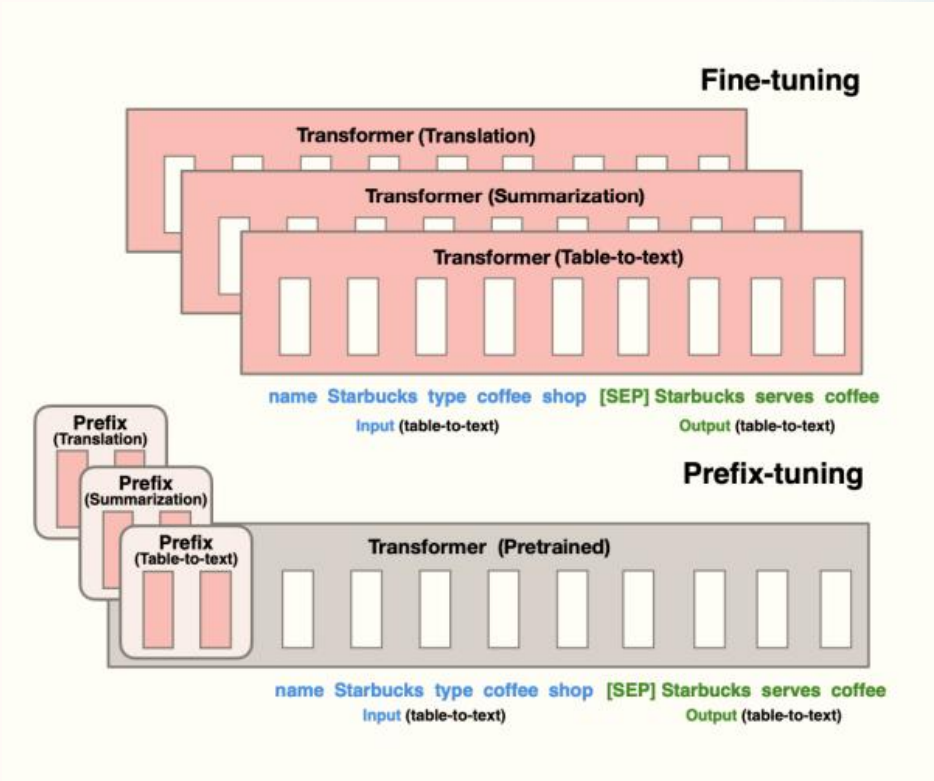
- Prefix-Tuning adds a **prefix** of parameters and **freezes all pretrained parameters.**

$K_{new} = [P_K ; K_{input}]$

$V_{new} = [P_V ; V_{input}]$

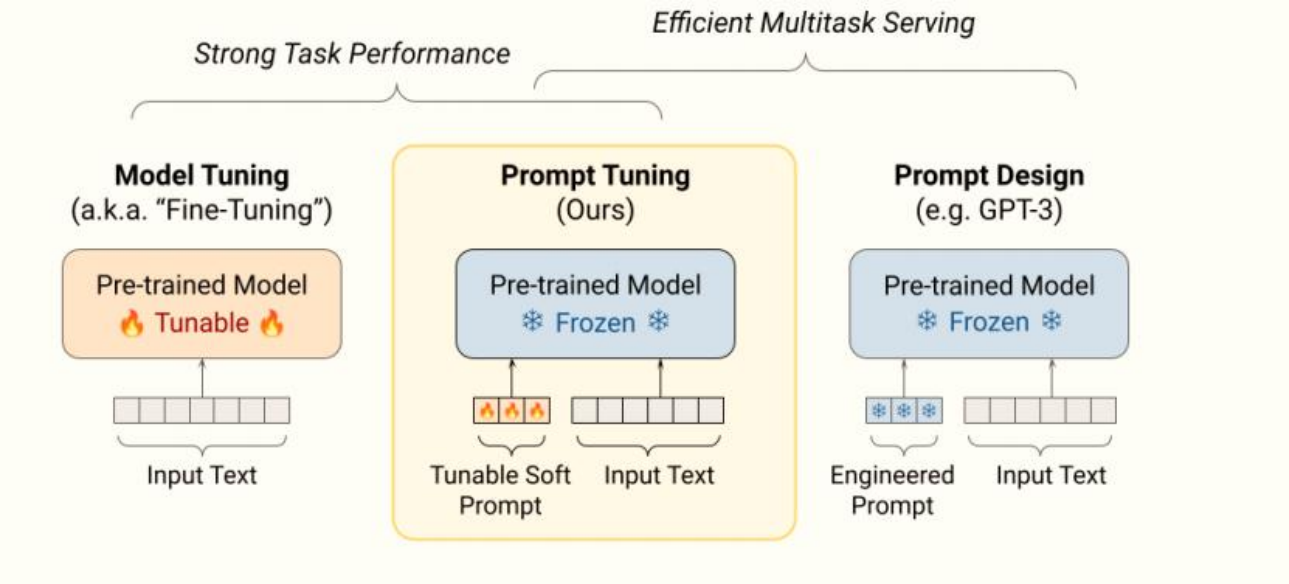
- The prefix is a sequence of continuous task-specific vector and is processed by the model just like real words would be, i.e., **"virtual tokens".**

- **Advantage:** each element of a batch at inference could run a different tuned model.



Prompt-Tuning

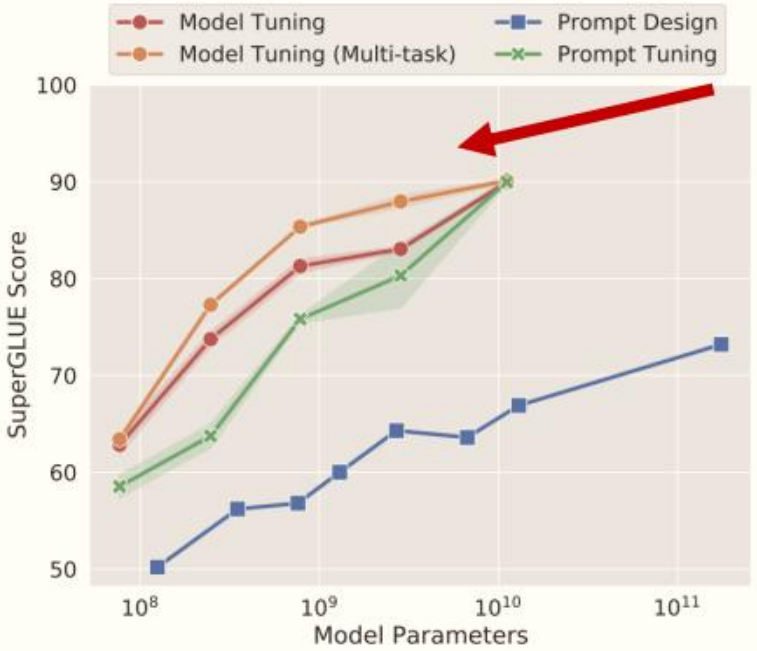
- Learning “soft prompts” to condition frozen LMs to perform downstream tasks
  - Prepend **virtual tokens to input**, and learn embeddings of these special tokens only



Prompt tuning only works well at scale

- Standard model tuning achieves strong performances but requires scoring separate copies of model for each end task

- Prompt tuning matches the quality of model tuning as size increases



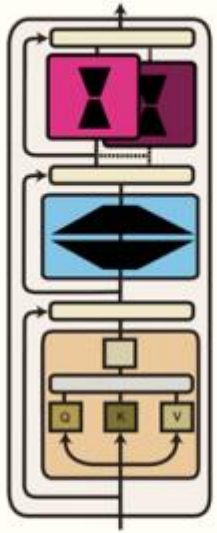
Lester, Brian, Rami Al-Rfou, and Noah Constant. "The power of scale for parameter-efficient prompt tuning." arXiv preprint arXiv:2104.08691 (2021).

A functional perspective of adaptation

• Function composition augments a model's functions with new task-specific functions:

$$f'_i(\boldsymbol{x}) = f_{\theta_i}(\boldsymbol{x}) \odot f_{\phi_i}(\boldsymbol{x})$$

• Most commonly used in multi-task learning where modules of different tasks are composed.

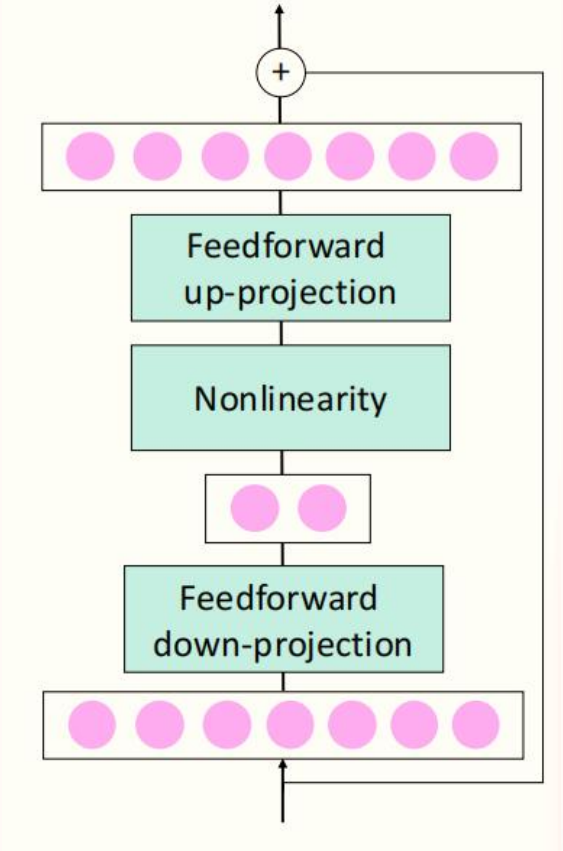


Function Composition

Adapter

• Insert a new function fϕ between layers of a pre_ trained model to adapt to a downstream task --- known as “adapters”

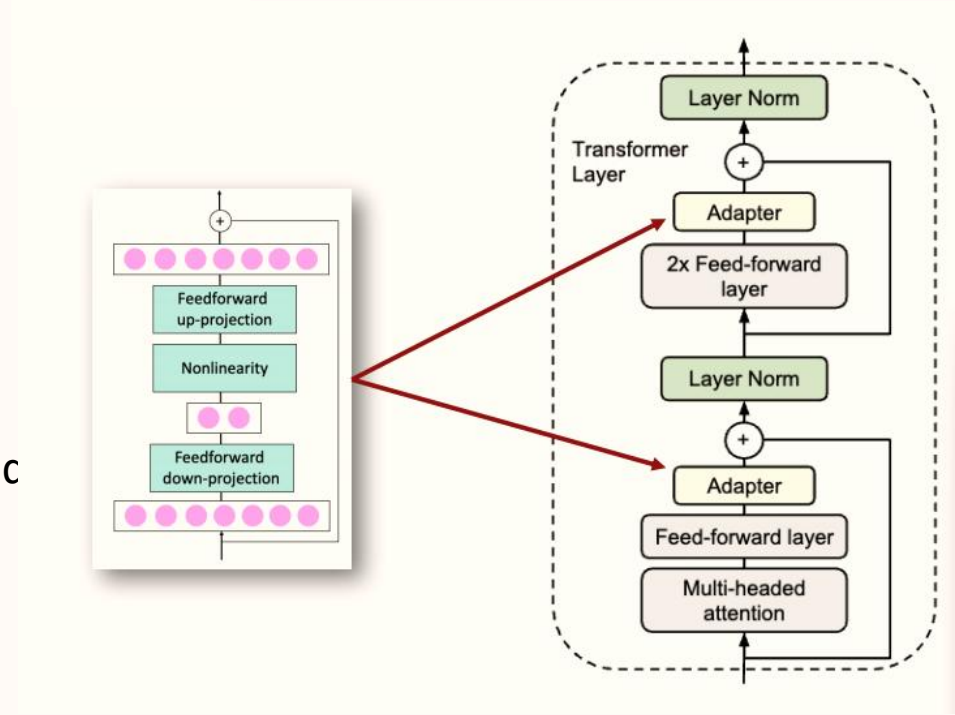
• An **adapter** in a Transformer layer consists of:
  • A feed-forward down-projection $W_D \in \mathbb{R}^{k \times d}$
  • A feed-forward up-projection $W_U \in \mathbb{R}^{d \times k}$
  • $f_\phi(\boldsymbol{x}) = W_U\big(\sigma(W_D \boldsymbol{x})\big)$



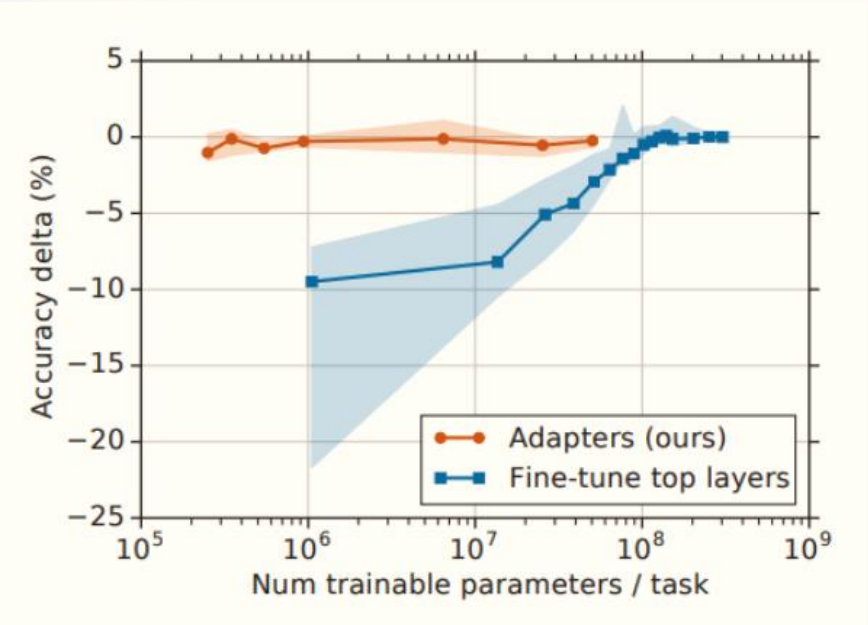
Adapter

- The adapter is usually placed after the multi-head attention and/or after the feedforward layer

- Most approaches have usec this bottleneck design with linear layers



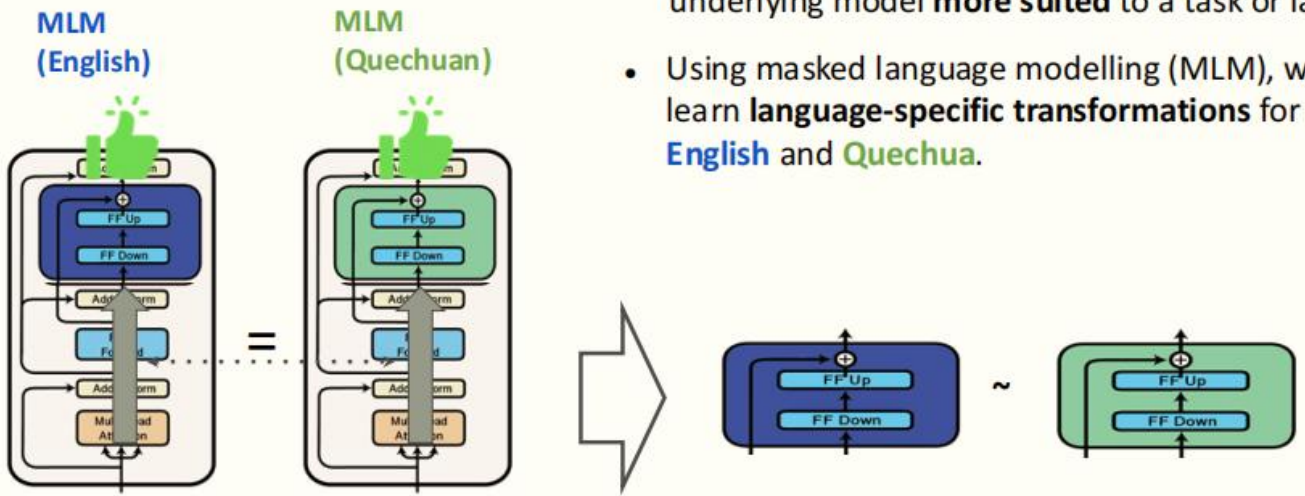
Adapter



The curves show the 20th, 50th, and 80th performance percentiles across nine tasks from the GLUE benchmark.

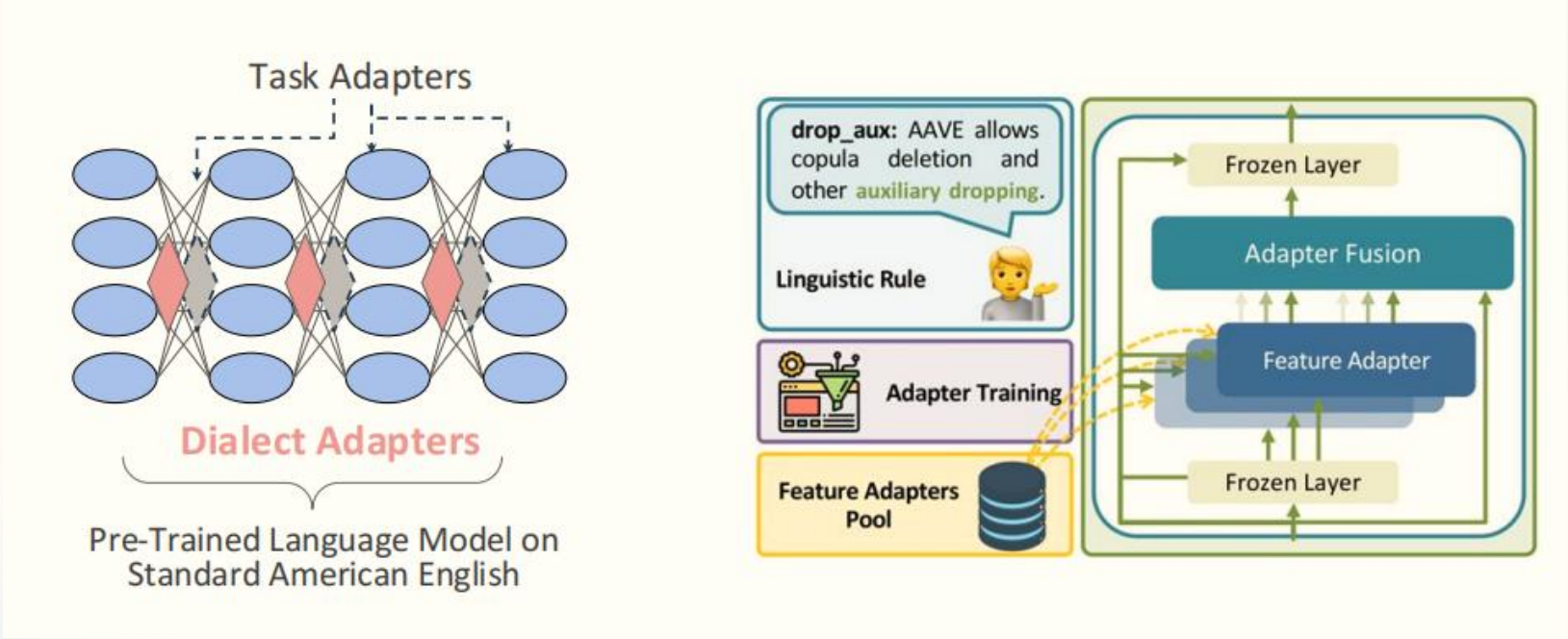
Adapter based tuning attains a similar performance to full finetuning with two orders of magnitude fewer trained parameters

Trade-off btw accuracy and # of trained task specific parameters



- Adapters **learn transformations** that make the underlying model **more suited** to a task or language.
- Using masked language modelling (MLM), we can learn **language-specific transformations** for e.g. English and Quechua.

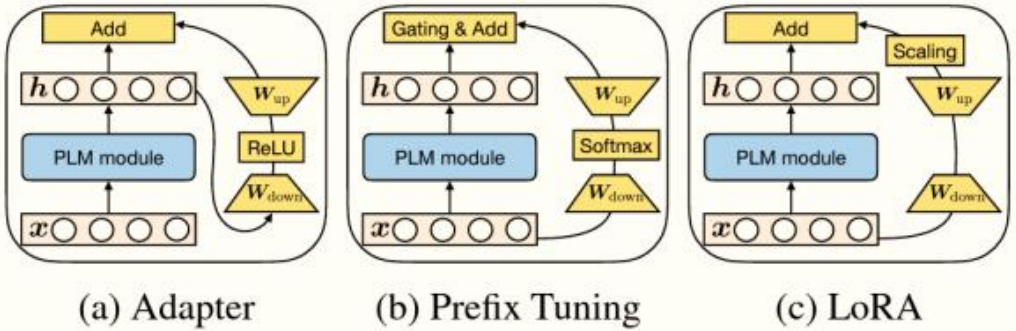
Using adapters for English dialect adaptation



Adapting LLMs trained on Standard American English to different English dialects (Held et al., 2023; Liu et al., 2023)

Unifying View

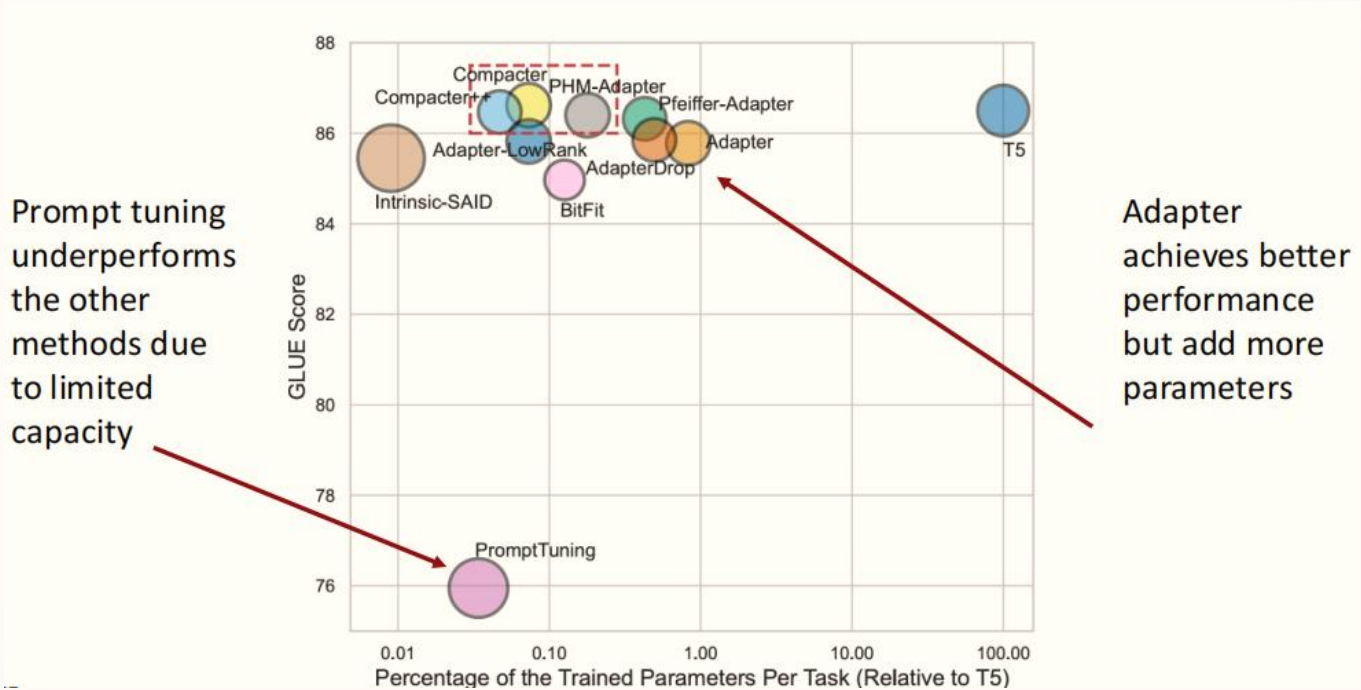
• He et al. show that LoRA, prefix tuning, and adapters can be expressed with a similar functional form
• All methods can be expressed as modifying a model's hidden representation $\boldsymbol{h}$



(a) Adapter (b) Prefix Tuning (c) LoRA

• Sparsity, structure, low-rank approximations, rescaling, and other properties can also be applied and combined in many settings

Performance comparison



2.

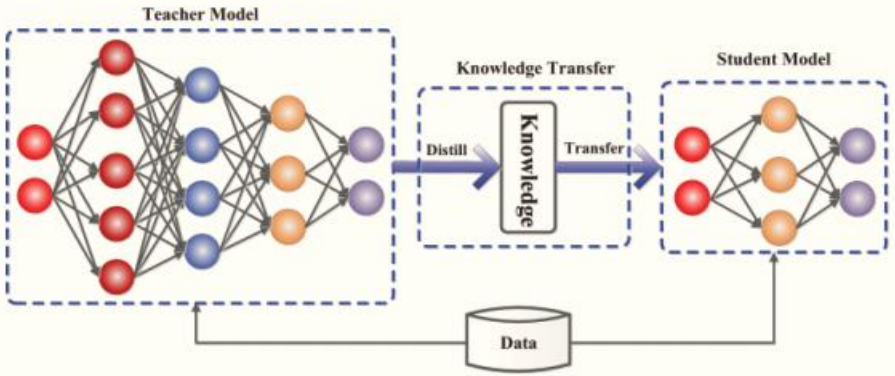
Knowledge distillation

Main Resource For this Section:

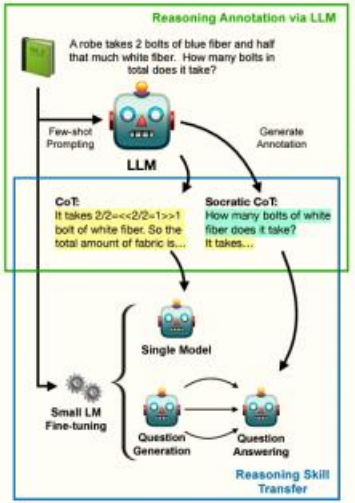
Advanced Natural Language Processing / Spring 2026 – Carnegie Mellon University

…Other variants of (efficient) adaptation

- **Knowledge distillation** to obtain smaller models



The generic teacher-student framework for knowledge distillation (Gou et al.,)

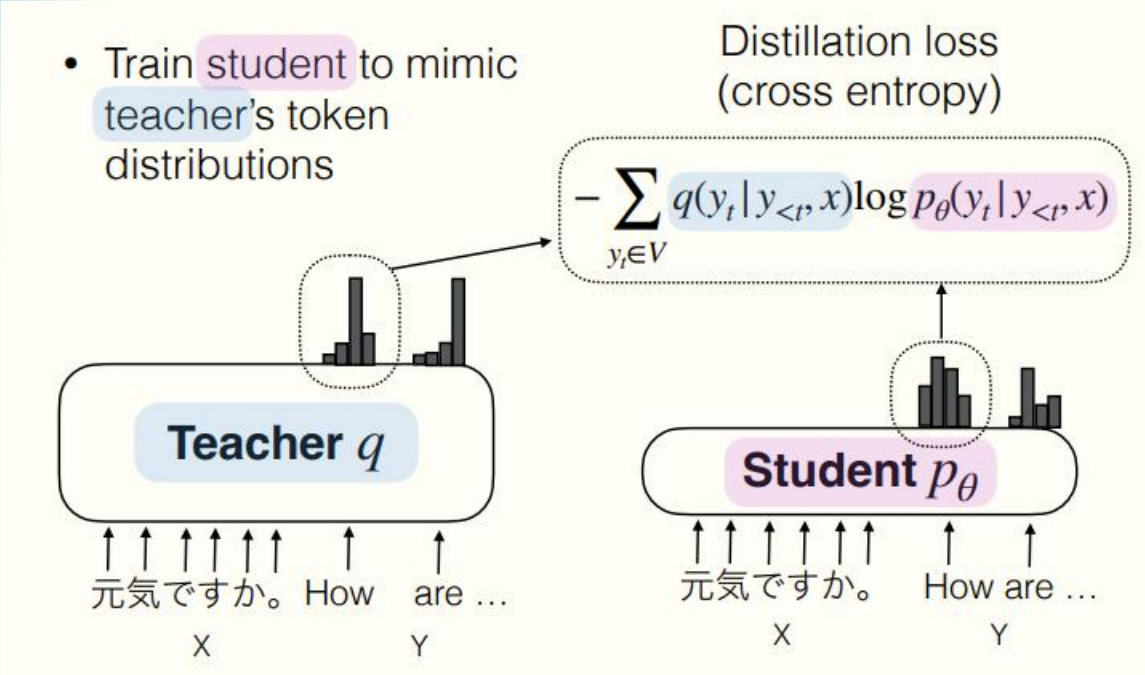


Shridhar et al.

- **Also check out**: Gist tokens (Wu), ReFT(Wu), etc

Token-level knowledge distillation

[Hinton et al 2015]



Token-level knowledge distillation [Hinton]

- Minimizes KL between teacher and student:

$$\min_{\theta} KL\left(q(y\,|\,x)\,\|\,p_\theta(y\,|\,x)\right)$$

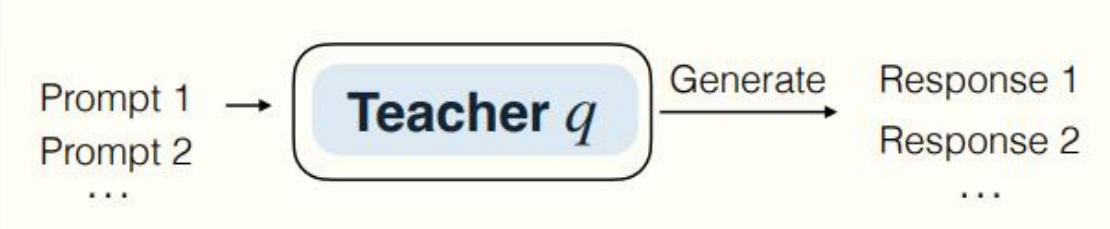
$$\equiv \min_{\theta} \mathbb{E}_{y\sim q(y|x)}\left[\sum_{t}\sum_{y_t\in V} -q(y_t\,|\,y_{<t},x)\log p_\theta(y_t\,|\,y_{<t},x)\right]$$

"Soft labels"

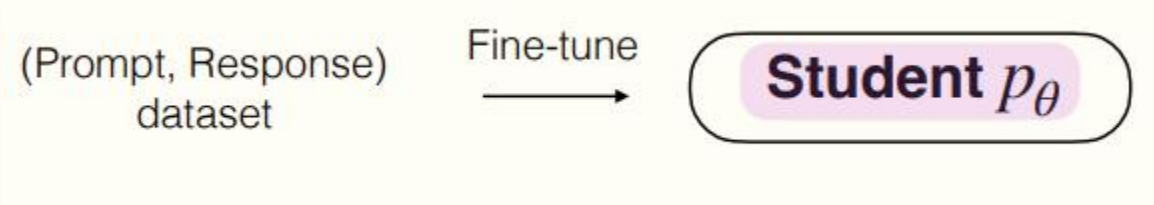
Sequence-level knowledge distillation [Kim & Rush]

Example: DeepSeek-R1-Distill-Qwen-7B

- Generate with a teacher model



- Student model fine-tunes on the generated data



Sequence-level knowledge distillation [Kim & Rush]

- Also minimizes KL between teacher and student:

$$\min_{\theta} KL\left(q(y \mid x) \| p_{\theta}(y \mid x)\right)$$

$$\equiv \min_{\theta} \mathbb{E}_{y \sim q(y|x)}\left[-\log p_{\theta}(y \mid x)\right]$$

- the teacher can be an “augmented” language model, e.g.

$$q \propto pLLM(y \mid x) \cdot A(x, y)$$

E.g. a classifier, verifier

- In principle, if the augmented teacher is better than , pLLM then the student can become better pLLM than through distillation

Exposure Bias & Capacity Gap

Distribution Shift (Exposure Bias):

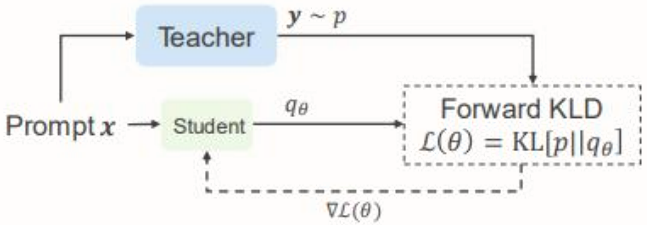
Standard sequence-level KD trains the student on teacher-generated outputs. During real-world autoregressive generation, early student errors compound rapidly, shifting inputs out of the teacher's distribution.

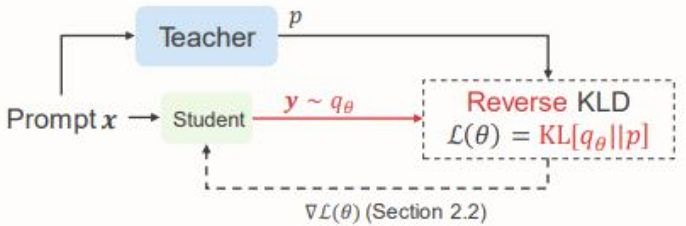
Capacity Mismatch:

A compact student model cannot represent the teacher's complex multimodal distribution. Forcing full coverage leads to uncertainty and hallucination.

Forward KL vs. Reverse KL

- **Traditional Forward KL: KL(p_teacher || p_student)**
  - **Zero-Avoiding Behavior:** Compels the student to place non-zero probability mass across every region where the teacher has mass.
  - **Consequence:** Stretches the student's limited capacity over low-confidence regions, causing distribution shift, over-generalization, and hallucination.
- **MiniLLM's Reverse KL: KL(p_student || p_teacher)**
  - **Mode-Seeking Behavior:** Incentivizes the student to focus strictly on major, high-confidence modes of the teacher.
  - **Consequence:** The student only generates text within high-confidence teacher regions, drastically reducing out-of-distribution hallucinations.





Advanced NLP - Session 10 - Parameter-Efficient Fine-Tuning (PEFT: LoRA, QLoRA) & Distillation - IUST

Parameter-Efficient Fine-Tuning(PEFT) Knowledge Distillation